

Semantic Caching: From Exact Hits to Close Enough

Anonymized Author

Institution

City, Country

email@email.email

Abstract

The rapid adoption of large language models (LLMs) has created demand for faster responses and lower costs. Semantic caching, reusing semantically similar requests via their embeddings, addresses this need but breaks classic cache assumptions and raises new challenges. In this paper, we explore offline policies for semantic caching, proving that implementing an optimal offline policy is NP-hard, and propose several polynomial-time heuristics. We also present online semantic aware cache policies that combine recency, frequency, and locality. Evaluations on diverse datasets show that while frequency based policies are strong baselines, our novel variant improves semantic accuracy. Our findings reveal effective strategies for current systems and highlight substantial headroom for future innovation. All of our code is open source.

CCS Concepts

• Information systems → Search engine indexing.

Keywords

Semantic Caching, Embeddings, Large Language Models

ACM Reference Format:

Anonymized Author. 2026. Semantic Caching: From Exact Hits to Close Enough. In *Proceedings of (CIKM '26)*. ACM, New York, NY, USA, 18 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Modern large language models (LLMs) rely on embedding methods to encode the semantic meaning of user queries into high-dimensional vectors. These embeddings enable efficient comparison of queries, making them a cornerstone of natural language processing tasks, such as question answering and semantic search. LLMs have gained wide adoption in the last few years although they suffer from high computational cost, as well as significant consumption of energy, memory, storage, and communication bandwidth.

Caching is a known technique to alleviate such limitations, and has been applied successfully in a variety of domains. Recently there have been several attempts to apply caching to LLMs [4, 15, 36]. By encoding each query into an embedding and comparing it against cached embeddings, semantically similar requests are detected, which enables the system to return the previously computed answer,

avoiding redundant inference and reducing overall latency and compute cost. Most of these works adopt a naïve or ad-hoc approach to managing the cache, i.e., for deciding which embeddings should be inserted into the cache, and once the cache is full, which should be evicted to make room for new ones. This is despite evidence from other domains that clever cache management can significantly improve caching benefits [12, 30, 33].

In particular, it is known that different workloads may yield significantly different hit rates depending on the cache management policy chosen. This has led to a plethora of cache management policies [2, 18, 26] as well as to various adaptive schemes and learning based schemes. However, a universally optimal cache management policy remains an open research challenge.

Since semantically similar queries can be answered by the same response, despite the fact that syntactically they are not exactly the same, most LLM caching solutions rely on *semantic caching*, in which a query is a hit if the cache includes the reply to some query such that the embedding vectors of both queries are “close enough” under some metric such as L2 norm or cosine similarity.

The impact of this shift from exact match caching to semantic caching on the vast knowledge about caching and cache replacement policies is not clear a priori. Hence, in this work we take a systematic approach to studying LLM cache management and, in particular, what changes, if any, are needed when moving from exact matches to semantic caching.

1.1 Summary of Contributions

We show that the optimal clairvoyant caching algorithm, also known as Belady’s OPT or MIN, is no longer optimal in the context of semantic caching, and that computing the corresponding semantic clairvoyant solution, which we term VOPT, is NP-hard. To address this, we propose and analyze three polynomial-time clairvoyant heuristics: Clustered Relaxed Vector Belady (CRVB), Volume Greedy Relaxed Vector Belady (FGRVB), and Time Greedy Relaxed Vector Belady (RGRVB). We compare these clairvoyant approaches against a broad range of established online cache management policies, adapted for semantic caching, across nine publicly available workloads. Our results show that frequency-based algorithms generally outperform other online approaches, with our novel SphereLFU achieving the highest accuracy. Also, in most workloads, the clairvoyant algorithms significantly outperform online policies, suggesting that there is still substantial remaining opportunities for innovation.

2 Related Work

Recent works, such as GPTCache [4], have demonstrated the benefits of applying caching to LLM inference. GPTCache can be integrated with multiple embedding models, vector stores and cache

Permission to make digital or hard copies of all or part of this work for personal or institutional use is granted by ACM Publishing, provided that the copies are distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '26, Rome, Italy

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-XXXX-X/2026/06 <https://doi.org/XXXXXXX.XXXXXXX>

management schemes. It supports LRU, LFU, FIFO, and Random cache replacement by default.

MeanCache is another recent cache for LLMs [15], which applies lighter models than GPTCache. MeanCache utilizes LRU as cache replacement, at least by default. For most workloads we have experimented with in this work, LRU is not a good cache policy.

GPTSemCache also applies caching to avoid repeated LLM API invocations [34]. They report a reduction in API calls by up to 68.8% across various query categories, with cache hit rates ranging from 61.6% to 68.8%, while achieving high accuracy. GPTSemCache relies on Redis to store cached data without any hard cache size limit, and therefore does not consider any replacement strategy.

vCache [38] employs an online learning algorithm to estimate an optimal threshold for each cached prompt similarity bound. They assume an infinite cache size, rendering the replacement policy irrelevant. When the cache size matters, our work becomes complementary to vCache and can be integrated with it. Similarly, InstCache [45] and GenerativeCache [20] are additional LLM caching mechanisms that assume an infinite cache and therefore do not apply any replacement policy.

VectorQ [37] introduces an adaptive semantic prompt cache that introduces a dynamic similarity metric. It utilizes a ML-based sampling approach to learn embedding-specific threshold regions to increase cache hit rates and minimize irrelevant hits. Similar to vCache, VectorQ focuses primarily on optimal thresholding and hit accuracy, assuming an unconstrained cache size.

Minimizing the cost of a cache based LLM system by learning the expected cost of various queries processing time, and integrating it with the Least Expected Cost policy and the Greedy Dual-Size with Frequency (GDSF) replacement algorithm was studied in [44]. They assume an oracle that clusters semantically similar queries into equivalence groups, thereby ignoring the issue of having multiple cached vectors that can be considered a hit for the same query. In that sense, their work can be compared to our ClusterLFU and CRVB policies, which we have shown to be often suboptimal.

RAGCache [23] caches RAG contents. It applies a novel prefix-aware Greedy-Dual-Size-Frequency replacement policy, and compares it to GDSF, LRU and LFU.

Proximity [8] is another RAG cache project. The focus in Proximity is reducing the cache search time, which is obtained by dividing the cache into independent buckets, or sets, and applying a mapping mechanism like locality-sensitive hashing (LSH) that ensures that semantically similar vectors are placed in the same bucket. Then each bucket can be managed using its own policy. Currently, Proximity supports LRU and FIFO. Our work is therefore complementing to [8], and our results can be applied for better selection of the replacement policies of the buckets of Proximity.

We note that caching high-dimensional vectors for similarity search is not exclusive to NLP. It is also useful, e.g., in recommender systems, reverse image search, and biological sequence analysis.

3 Preliminaries

3.1 Semantic Caching

A semantic (vector) cache C is parameterized by (i) Dim - the dimensionality of each vector, (ii) N - the maximum number of vectors the cache can store, and (iii) D_{thresh} - a distance threshold. A stored

vector within this distance from a request is considered a *cache hit*. Each stored vector is a tuple of Dim floats. In our evaluation, all vectors are L2-normalized to have unit magnitude, although this is not strictly required. The cache exposes two basic methods:

Query: Given a vector V and an integer M , return the M or less vectors that are closest to V , restricted to those within a distance smaller than D_{thresh} .

Update: Given a vector V , the cache is requested to store V . If the cache already contains N vectors, inserting the new vector requires evicting one of the stored vectors from the cache. The cache's management policy decides which vectors are stored and which ones are evicted.

For efficiency reasons, our cache implementation supports batches, i.e., querying for multiple vectors using a single request and caching multiple vectors with a single request.

3.2 Embedding

Embedding methods aim to project high-dimensional textual data into vector representations that preserve semantic relationships. Sentence-BERT (SBERT) [35] is a widely used model of this kind, which offers a balanced tradeoff between semantic clustering quality and runtime efficiency. In our experiments, we normalize the embeddings so that their magnitude equals 1, and use the all-MiniLM-L6-v2 variant, which produces 384-dimensional vectors.

3.3 Embeddings Similarity

Embedding similarity is commonly measured using cosine similarity, with L2 distance serving as a closely related alternative [10].

In this work, we compute similarity using L2 distance. Our embeddings are normalized to have a magnitude of 1, therefore cosine similarity and L2 distance become equivalent up to a monotonic transformation, and the choice between them has no effect on the nearest neighbor results.

For normalized embeddings, the L2 distance directly reflects angular similarity. Identical embeddings have distance 0, while completely opposite embeddings have distance 2. A distance of $\sqrt{2}$ corresponds to orthogonal vectors, i.e., unrelated vectors. Distances between 0 and $\sqrt{2}$ indicate increasing alignment: for example, an euclidean distance of 0.5 corresponds to a cosine similarity of about 0.875, meaning the vectors are close but not identical. Conversely, distances beyond $\sqrt{2}$ indicate increasing opposition, with values approaching 2 representing strong dissimilarity.

In this work, we used three normalized L2 distance thresholds (D_{thresh}): 0.5, 0.7, and 0.9, which correspond to cosine similarity scores of approximately 0.88, 0.75, and 0.60, respectively. We selected these values based on established benchmarks: 0.88 targets strict near-duplicate detection, 0.75 serves as a balanced baseline for semantic equivalence, and 0.60 functions as a high-recall filter for broader topical relevance [9, 35].

4 Datasets

We have obtained nine real-world datasets and processed them into a unified format. They span chats, conversational reformulations, community Q&A, web search, open-domain QA, and general consumer questions. Our canonical datasets can be generated by

our code, available online [1]. We only embed queries from these datasets, ignoring responses or supporting documents.

ELI5: A collection of question–answer pairs drawn from the "Explain Like I'm Five" subreddit [13]. The questions are short, usually up to a sentence, and expressed in casual, colloquial language. They often reflect trending topics or popular opinions.

WildChat: Dialogues constructed from real ChatGPT logs, preserved in chronological order and grouped into sessions [43]. Users frequently resubmit identical or similar queries, either within the same session or across simultaneous sessions. This behavior induces a recency bias.

Natural Questions: An open-domain QA dataset designed around questions answerable using Wikipedia [27]. These questions are general and less influenced by conversational context, in contrast to session-oriented datasets.

MS MARCO: A large-scale dataset of real Bing search queries paired with relevant passages, released by Microsoft [3]. The queries mirror web-search style phrasing, ranging from keyword-style to natural-language questions.

StackOverflow: Programming-related questions obtained from the Google BigQuery public dataset of StackOverflow posts [16]. We restrict ourselves to question titles, which are typically concise, making them useful for evaluating embeddings on structured knowledge tasks.

Quora Question Pairs: A dataset of user-submitted questions on Quora, annotated for semantic equivalence [21]. This property makes the dataset particularly suitable for studying semantic overlap and redundancy.

MMLU: A benchmark covering 57 subjects, ranging from elementary mathematics to professional law and the humanities [17].

HotpotQA: A dataset focused on multi-hop reasoning, where questions cannot be answered by a single document alone but require combining information from multiple supporting sources [42].

TriviaQA: A large-scale reading comprehension dataset containing trivia questions, paired with evidence from web search results [25]. The questions are complex and often compositional.

These datasets rarely carry ordering signal, and are mostly affected by frequency. They follow a specific distribution: when calculating frequency as the number of neighbors of distance $< D$, regardless of the selected D , the frequency vs rank graph follows a Zipfian distribution. An expanded analysis of the datasets is available [online \[?\]](#).

4.1 Datasets Metrics

Table 2 presents key statistical properties of the embeddings (using the first 100K queries of each dataset) to characterize the geometric structure and clustering tendency of the semantic space. We consider the following metrics:

Cos Sim (Avg/Std): The average and standard deviation of cosine similarity between all pairs of embeddings. The low average values (near 0) indicate that the embedding space

is sparsely populated, and random queries are typically orthogonal to one another.

L2 (Mean/Std): The average Euclidean distance between all embeddings pairs. Since our embeddings are normalized to the unit sphere, orthogonal vectors have a distance of $\sqrt{2} \approx 1.414$. The observed means (≈ 1.37 – 1.40) align with the cosine similarity results, confirming that unrelated queries are far apart in the vector space.

PCA Ent. (Entropy): A measure of the effective dimensionality of the data, calculated as the entropy of the explained variance ratio of the PCA components. A higher value indicates that information is distributed across many dimensions, whereas a lower value implies the data lies on a lower-dimensional manifold.

Cluster (AVG/STD): We group vectors into semantic clusters using the threshold $D_{\text{thresh}} = 1$. **Cluster AVG** denotes the average size of a cluster. values near 1.0 (e.g., StackOverflow) indicate high uniqueness, while higher values (e.g., MMLU) indicate high redundancy. **Cluster STD** a high standard deviation (e.g., WildChat) indicates a long tail distribution where specific topics are extremely popular while most others are unique.

Hopkins Statistic: A value of 0.5 indicates a uniform (random) distribution, while values approaching 1.0 indicate a tendency to cluster. The scores (ranging from 0.74 to 0.88) confirm that the datasets possess significant semantic structure, justifying the use of semantic caching.

5 Theoretical Analysis

5.1 OPT

OPT, also called Belady's optimal or MIN, is the clairvoyant cache policy that, upon a miss with a full cache, evicts the resident item whose next request occurs farthest into the future [6]. This rule provably minimizes total misses for any fixed capacity on a given finite request trace. Classic exchange-argument proofs show that any deviation cannot improve the miss count. Because it requires perfect future knowledge, OPT is not implementable online, but it serves as an oracle upper bound for evaluation and as a target for online policies. Computing OPT for a cache sized N and a known in advance trace of length n takes $O(n \log N)$. OPT defines the achievable frontier against which real policies are compared.

As stated before, in semantic caching, a hit occurs if the requested vector lies within D_{thresh} of a cached vector. This heuristic breaks OPT's optimality: consider a workload with two disjoint regions of requests, one dense and popular and the other sparse and unpopular. An OPT policy translated directly would fill the cache with vectors from the dense region, although only a handful are needed to cover that region. On the other hand, the sparse region will suffer unnecessary misses. This means we must redefine an optimal algorithm for semantic caching.

5.2 VOPT

We define VOPT as an offline policy that achieves the maximum hit rate for a given semantic cache and workload. Different policies may achieve the maximum hit rate, and we would consider all of them to be instances of VOPT.

	MsMarco	WildChat	ELI5	NQ	SO	Quora	MMLU	TriviaQA	HotPotQA
Cos Sim Avg	0.0182	0.0484	0.0523	0.0206	0.0170	0.0451	0.0562	0.0570	0.0848
Cos Sim Std	0.0757	0.0861	0.0845	0.0871	0.0893	0.0843	0.0995	0.0919	0.1011
L2 Mean	1.4019	1.3781	1.3754	1.3978	1.4019	1.3806	1.3718	1.3715	1.3508
L2 Std	0.0555	0.0655	0.0633	0.0644	0.0658	0.0635	0.0759	0.0687	0.0766
PCA Ent.	7.8781	7.8251	7.7055	7.6928	7.6043	7.7551	7.5655	7.5658	7.4615
Cluster AVG	1.0203	1.3606	1.0434	1.2347	1.0077	1.2958	1.9112	1.0716	1.0081
Cluster STD	0.1491	2.5079	0.2383	0.6622	0.0896	0.8880	1.3860	0.2975	0.0925
Hopkins	0.7401	0.8044	0.7558	0.7897	0.7424	0.8054	0.8839	0.7551	0.7252

Table 1: Per dataset metrics for first 100K embeddings in each dataset.

THEOREM 5.1 (HARDNESS OF COMPUTING VOPT). Let $D_{\text{thresh}} > 0$ and let N be the cache capacity. Given a request sequence of vectors, computing VOPT is NP-hard.

PROOF. We show a reduction from the Maximum Coverage Problem (MCP). In MCP, we are given sets $S_1, \dots, S_m$ over a ground set $U = \{e_1, \dots, e_n\}$ and an integer N . The goal is to choose at most N of the sets whose union covers the largest number of elements [19].

Requests. We construct vectors u_S (sets) and v_e (elements) in $\mathbb{R}^{m+n}$ in polynomial time. For distinct sets S, S' and elements e, e' :

- (1) $d(u_S, u_{S'}) > D_{\text{thresh}}$ and $d(v_e, v_{e'}) > D_{\text{thresh}}$.
- (2) $d(u_S, v_e) \leq D_{\text{thresh}} \iff e \in S$.

Vector Construction. Let the MCP instance consist of a universe of elements $U = \{e_1, \dots, e_n\}$ and a collection of sets $\mathcal{S} = \{S_1, \dots, S_m\}$. Let $K_{\max} = \max_j |S_j|$ be the size of the largest set. When $K_{\max} = 1$, the problem is trivial. W.l.o.g., we assume $K_{\max} > 1$. We define a Euclidean space $\mathbb{R}^{n+m}$ with orthonormal basis vectors:

- $\{u_1, \dots, u_n\}$ corresponding to the n elements.
- $\{w_1, \dots, w_m\}$ corresponding to the m sets.

We construct the vectors using scaling parameters $\alpha, \beta, \gamma > 0$. For each element $e_i \in U$, we define the *element vector*: $v_{e_i} = \alpha \cdot u_i$.

For each set $S_j \in \mathcal{S}$, we define the *set vector* as a weighted combination of its members and a unique identifier:

$$v_{S_j} = \sum_{e_k \in S_j} (\beta \cdot u_k) + (\gamma \cdot w_j) \quad (1)$$

For the reduction to hold, the embedding must satisfy four conditions relative to the cache threshold D_{thresh} :

- (1) **Element Separation:** Distinct elements must not cover each other.
- (2) **Set Separation:** Distinct sets must not cover each other.
- (3) **Membership Coverage:** A set must cover its member elements.
- (4) **Non-Membership Separation:** A set must not cover non-member elements.

These conditions correspond to the following system of inequalities regarding the squared Euclidean distance:

$$\|v_{e_i} - v_{e_\ell}\|^2 = 2\alpha^2 > D_{\text{thresh}}^2 \quad \forall i \neq \ell \quad (2)$$

$$\|v_{S_j} - v_{S_\ell}\|^2 \geq 2\gamma^2 > D_{\text{thresh}}^2 \quad \forall j \neq \ell \quad (3)$$

$$\|v_{S_j} - v_{e_i}\|^2 \leq D_{\text{thresh}}^2 \iff e_i \in S_j \quad (4)$$

Expanding the distance for a set S_j and an element e_i , we distinguish two cases:

- **Case 1** ($e_i \in S_j$): The distance is minimized due to the shared component β : $(\alpha - \beta)^2 + (|S_j| - 1)\beta^2 + \gamma^2 \leq D_{\text{thresh}}^2$.
- **Case 2** ($e_i \notin S_j$): The components are orthogonal, maximizing distance: $\alpha^2 + |S_j|\beta^2 + \gamma^2 > D_{\text{thresh}}^2$.

We now prove that valid parameters α, β, γ always exist. We set β such that the set vector "leans" towards its members relative to the element scale: $\beta = \frac{\alpha}{K_{\max}}$.

We analyze the constraints for the "worst-case" scenario (a set of maximum size $K_{\max}$). Substituting β , the squared distance for a member element becomes:

$$\text{Dist}_{\text{in}}^2 = \alpha^2 \left(1 - \frac{1}{K_{\max}} \right) + \gamma^2 \quad (5)$$

To satisfy all conditions, we must find α and γ such that:

$$2\alpha^2 > D_{\text{thresh}}^2 \quad 2\gamma^2 > D_{\text{thresh}}^2 \quad \alpha^2 \left(1 - \frac{1}{K_{\max}} \right) + \gamma^2 \leq D_{\text{thresh}}^2 \quad (6)$$

From (6), we derive an upper bound for γ^2 :

$$\gamma^2 \leq D_{\text{thresh}}^2 - \alpha^2 \left(\frac{K_{\max} - 1}{K_{\max}} \right) \quad (7)$$

Further, this upper bound must strictly exceed $\frac{D_{\text{thresh}}^2}{2}$:

$$D_{\text{thresh}}^2 - \alpha^2 \left(\frac{K_{\max} - 1}{K_{\max}} \right) > \frac{D_{\text{thresh}}^2}{2} \quad (8)$$

Rearranging for α^2 , we obtain the existence condition:

$$\alpha^2 < \frac{D_{\text{thresh}}^2}{2} \cdot \left(\frac{K_{\max}}{K_{\max} - 1} \right) \quad (9)$$

Since $K_{\max} \geq 2$ implies $\frac{K_{\max}}{K_{\max} - 1} > 1$, there exists a non-empty open interval for α^2 : $\frac{D_{\text{thresh}}^2}{2} < \alpha^2 < \frac{D_{\text{thresh}}^2}{2} \cdot \left(\frac{K_{\max}}{K_{\max} - 1} \right)$.

Choosing any α^2 within this interval and setting γ^2 sufficiently close to (but greater than) $D_{\text{thresh}}^2/2$, satisfies all strict inequalities. This guarantees that a valid embedding exists for any MCP instance. $\square$

Request sequence. We define a two-phase request sequence:

- **Phase 1:** Request all set vectors $u_{S_1}, \dots, u_{S_m}$. All requests are misses, but the policy may keep up to N set vectors in the cache after Phase 1.
- **Phase 2:** Request all element vectors $v_{e_1}, \dots, v_{e_n}$. A request to v_{e_i} is a hit iff some cached set vector u_{S_j} corresponds to a set containing e_i .

Correctness. The number of hits in Phase 2 equals the number of elements in the union of the N sets associated with the cached set vectors. Thus, an optimal VOPT policy chooses the same N sets that achieve optimal maximum coverage. Since MCP is NP-hard, so is computing VOPT. $\square$

COROLLARY 5.2 (INAPPROXIMABILITY OF VOPT). *Unless $P = NP$, no polynomial-time algorithm can approximate VOPT's hit ratio within a factor better than $(1 - 1/e)$.*

PROOF. The reduction in Theorem 5.1 is approximation-preserving: the optimal number of Phase 2 hits equals the maximum coverage value of the MCP instance. MCP is known to be NP-hard to approximate to within a factor better than $(1 - 1/e)$ [19]. Thus, any algorithm approximating VOPT more closely would imply an algorithm approximating MCP more closely, contradicting known hardness results. $\square$

THEOREM 5.3 (WELL-DEFINEDNESS AND COMPUTATIONAL INFEASIBILITY OF VOPT). *For any request sequence and threshold distance D_{thresh} , VOPT is well-defined: an optimal selection of up to N cached vectors that maximizes semantic hits always exists. However, computing VOPT requires solving an NP-hard optimization problem and is therefore computationally infeasible for realistic workloads.*

PROOF. Well-definedness follows from the fact that the request sequence is finite and there are finitely many subsets of size at most N that the cache may contain. Each such subset induces a unique number of semantic hits, and an optimal choice exists. $\square$

COROLLARY 5.4 (APPROXIMABILITY OF THE STATIC FORMULATION). *Consider the static variant of the problem where the cache contents S ($|S| \leq N$) are selected and inserted prior to processing the request sequence and remain fixed. In this setting, the objective function is monotone submodular, and a greedy selection strategy achieves an approximation ratio of $(1 - 1/e) \approx 0.632$.*

The reduction from the Maximum Coverage Problem (MCP) in Theorem 5.1 reveals a fundamental structural property: if the cache admits no evictions (i.e., the content is static), the problem of maximizing hits is mathematically equivalent to MCP. The objective function of MCP exhibits diminishing returns, a property formally known as submodularity. It is known [31] that for such problems, iteratively selecting the element that covers the most uncovered items yields the optimal $(1 - 1/e)$ approximation.

However, this strict theoretical guarantee does not extend to the general (dynamic) semantic caching problem. In dynamic settings, an eviction decision at time t constrains the feasible cache states at time $t + 1$, creating temporal dependencies that break the submodularity of the global objective function. Consequently, while the $(1 - 1/e)$ approximation is no longer strictly guaranteed, the static relaxation serves as the theoretical foundation for our offline heuristics. By treating the dynamic problem as a sequence of local maximum coverage problems, where we attempt to maximize the number of future hits covered by the current cache state, we leverage the strong performance of greedy strategies on submodular functions, even in the absence of worst-case guarantees.

COROLLARY 5.5 (SUFFICIENCY OF CLIQUE COVER FOR OFFLINE OPTIMALITY). *Let G be the semantic intersection graph of a request*

sequence R . A cache with capacity $N \geq \theta(G)$, where $\theta(G)$ is the minimum clique cover number of G , is sufficient to achieve the maximum possible hit rate. This is achieved by storing at least one representative vector from each clique in a minimum clique cover.

5.3 VOPT heuristics

We propose 3 heuristic algorithms for VOPT. In our measurements, as reported below, we have found that different ones obtain slightly higher or lower hit rate based on the workload and cache size. Obviously, since these are offline schemes, meant to establish an upper bound on the best attainable performance, and they are computationally efficient, one can simply implement all three and take the best one at any given data point.

5.3.1 Clustered Relaxed Vector Belady. As mentioned before, the challenge in adapting OPT to semantic caches is that multiple cached vectors may cover the same future requests, leading to coverage redundancy. One method to utilize our clairvoyant knowledge of all requests is to divide them into semantically identical clusters, where the maximum distance between two vectors in the same cluster is D_{thresh} . To create the clusters, we first construct a graph G in which each node represents one embedding vector. An edge is added between two nodes if their embeddings are semantically identical, i.e., if their distance is below the threshold D_{thresh} .

Since any cluster of semantically identical embeddings corresponds to a clique in G , the task of minimizing the number of clusters reduces to finding the Minimum Clique Cover (MCC). This problem is NP-hard, and can be solved by coloring the complement graph of G , denoted $\bar{G}$ [14]. A useful heuristic is greedy coloring with DSATUR, which strikes a balance between accuracy and runtime. However, we found that for this use case, the complement graph is too large and dense for the coloring method to be practical. Instead, we adopt a simple greedy approach: iteratively extracting maximal cliques. While this method yields lower quality partitions, it produces sufficiently good results in significantly lower runtime, particularly when the graph is sparse as in our case.

After clustering, the problem reduces to classic caching with exact matching, where each embedding is represented by its cluster ID. In this setting, OPT is provably the optimal policy and thus achieves the highest possible hit rate among policies that rely solely on clusters. If embeddings form true equivalence classes, i.e., if the similarity relation is transitive ($d(v_i, v_j) \leq D_{\text{thresh}} \wedge d(v_i, v_k) \leq D_{\text{thresh}} \Rightarrow d(v_j, v_k) \leq D_{\text{thresh}}$), then clustering perfectly captures semantic identity, and CRVB yields optimal results. A further consequence of this property is that finding the Minimum Clique Cover becomes trivial, since each connected component is a clique.

In practice, it occurs that a vector v_i is semantically equivalent to both v_j and v_k , but v_j and v_k are not equivalent. Thus, CRVB is not optimal in the general case, and is only used as a baseline oracle.

5.3.2 Frequency Greedy Relaxed Vector Belady. FGRVB attempts to approximate the optimal policy by focusing on the quantity of future hits. This is motivated directly by the reduction to MCP.

Although we cannot guarantee the $(1 - 1/e)$ bound in the dynamic setting, FGRVB implements the greedy heuristic for submodular maximization as a continuous eviction policy. FGRVB ranks

cached embeddings by their marginal contribution to the total number of future hits. On a cache miss, FGRVB counts the number of future unique hits attributed to the missed request, i.e., hits that are only covered by said embeddings. It replaces the cached item with the least unique covered future hits, given that it covers fewer hits compared to the missed item.

Let $R_t = (r_t, r_{t+1}, \dots)$ be the sequence of all future requests. For a cached vector v , we define its volume score as the cardinality of the subset of future requests it covers:

$$\text{Vol}(v) = |\{r_i \in R_t : d(v, r_i) < D_{\text{thresh}}\}| \quad (10)$$

To handle redundancy, a strict implementation calculates the *marginal gain*, i.e., how many future requests v covers that are not already covered by other cached items. Upon a cache miss with a full cache, FGRVB evicts the vector with the lowest marginal volume score. In essence, the policy attempts to maintain the set of N vectors that "cover" the maximum possible number of remaining requests in the trace, regardless of when those requests occur.

5.3.3 Recency Greedy Relaxed Vector Belady. To address the fact that vectors from one cluster may also cover vectors in another, we developed RGRVB. The core idea is based on the cover relation, which expresses the potential future hits of an embedding vector:

$$\text{cover}(r_i) = \{r_j : d(r_i, r_j) < D_{\text{thresh}}, j > i\},$$

where $d(\cdot, \cdot)$ denotes the embedding distance. For each embedding vector r_i , following request number t , we define

$$\text{NextCover}(r_i, t) = \min\{j : j > t, r_j \in \text{cover}(r_i)\},$$

with $\text{NextCover}(r_i, t) = \infty$ if no such j exists. Cache cover is then

$$\text{CacheCover}(C, t) = \{\text{NextCover}(r_j, t) \mid r_j \in C\}.$$

While the cache is not yet full, every embedding vector is admitted. After the cache reaches capacity, a new embedding r_i is accepted only if its next cover is not already served:

$$\text{NextCover}(r_i, t) \notin \text{CacheCover}(C, t).$$

If admitted, r_i replaces the embedding vector whose next hit lies furthest in the future. We consider the next hit rather than all future hits; while embeddings may cover requests far ahead, by the time those requests occur the cache has changed enough to make such predictions unreliable. Also, empirically, focusing on the next hit yields better results.

6 Online Policies

6.1 Adaptions of Classic (Exact Match) Caching

As mentioned before, numerous cache management policies have been developed over the years, e.g., [2, 5, 11, 12, 18, 22, 24, 26, 28–30, 32, 33, 39–41]. Here, we briefly elaborate on the ones we have experimented with in this work, which were chosen based on their popularity and as each represents a class of similar policies.

LFUDA. LFUDA [2] enhances LFU by introducing a dynamic aging mechanism to prevent cache pollution by stale but historically popular items. It maintains a global aging factor that increases over time, ensuring that older items with high access counts are eventually evicted if not accessed recently. This approach balances

between recency and frequency, adapting better to changing workloads compared to standard LFU. LFUDA is particularly useful in long running systems where workload patterns evolve over time.

LRU-K. LRU-K [32] is an extension of the LRU policy that considers the K -th most recent access when making eviction decisions. By doing so, it captures long term access patterns rather than reacting solely to the most recent usage. When $K = 1$, LRU-K reduces to standard LRU. As K increases, the policy becomes more resistant to short-term fluctuations, favoring items that exhibit consistent reuse over time. However, LRU-K requires additional space to maintain the history of accesses, and its performance depends on the appropriate choice of K .

Adaptive Replacement Cache (ARC). ARC [30] dynamically balances between recency and frequency by maintaining two LRU lists: one for recently accessed entries and one for frequently accessed entries. It uses a self tuning mechanism to adapt to workload characteristics by adjusting the size of each list based on observed hit rates. This adaptability allows ARC to outperform both LRU and LFU in many scenarios without requiring manual parameter tuning. ARC is particularly effective in systems where access patterns shift between recency and frequency dominated behaviors. LeCaR [41], a well known ML based cache policy, demonstrated hit rates comparable to ARC in its original paper.

Random Admission Policy (RAP). RAP [7] was originally proposed for top- k and frequency estimation, maintaining counters for the most frequent items in a stream when the stream size far exceeds available memory. We adapt it for cache management. Similar to LFU, RAP identifies the least frequently used item for potential replacement. However, instead of always evicting it, the item with counter value C is replaced with probability $\frac{1}{C+1}$. This probabilistic admission reduces unnecessary evictions caused by infrequent (tail) items, preserving space for items more likely to be reused.

6.2 Novel Cache Eviction Policies

As we mentioned in the Introduction, caching embedding vectors for LLM workloads presents unique challenges in applying classical cache management policies. Specifically, embedding vectors encode semantic similarity in a continuous embedding space, which allows for "close enough" hits. However, this complicates eviction decisions. For example, two distinct queries may map to nearly identical embeddings, making it redundant to store both. Similarly, groups of related embeddings may exhibit correlated access patterns, suggesting that decisions should sometimes be made at the cluster level rather than the individual vector level. To address these challenges, we introduce several novel eviction policies that exploit semantic structure, vector similarity, and linguistic properties of the underlying data, in order to evaluate how to best exploit the properties of embedding vectors.

6.2.1 MissLFU. Similar to LFU, but only inserts an embedding vector into the cache if there are no stored semantically identical vectors in the cache. The intuition here is that if a vector is close enough to an existing vector in the cache, there is already a (semantic) hit, so there is no point inserting the new vector. Hence,

we prefer to keep embedding vectors that may otherwise not have a cached semantically close representative.

6.2.2 ClusterLFU. This policy groups vectors into clusters using a greedy online clustering algorithm (although other clustering algorithms can be used as well). It then applies LFU at the cluster level rather than for individual vectors. Each cluster maintains a single LFU counter representing the aggregate access frequency of its members. When an eviction is required, the least frequently used cluster is selected, and a random vector from that cluster is removed. By aggregating counters at the cluster level, this approach reduces metadata overhead and exploits semantic similarity among vectors, allowing the cache to favor groups of related embeddings that are accessed frequently.

6.2.3 ClusterLRU. Similar to ClusterLFU, this policy operates at the cluster level using the same greedy online clustering method. Instead of tracking frequency, it maintains the most recent access time for each cluster, based on the latest access to any of its members. Upon eviction, the least recently used cluster is selected, and a random vector from that cluster is removed. This aggregation reduces the need to maintain per-vector recency data, making it more efficient for large vector caches while capturing temporal locality.

6.2.4 DistanceLFU. This policy adjusts LFU counting based on vector similarity to incoming queries. If the closest stored embedding to a query is at distance d (with D_{thresh} being the maximum distance threshold to be considered a hit), the corresponding counter is incremented by: $\Delta = 1 - \frac{d}{D_{\text{thresh}}}$.

This weighting increases the counter proportionally to how close is the match, allowing the cache to prioritize embeddings that better match recent queries.

6.2.5 Surprisal. This policy leverages linguistic surprisal to guide eviction decisions. Surprisal measures how unexpected a sentence is, with higher values indicating lower probability of occurrence. Perplexity is a metric closely related to surprisal, and is defined as the inverse probability of the sequence normalized by its length.

Direct computation of perplexity from an LLM is computationally expensive for online caching. Instead, a more efficient metric can be obtained using a precomputed dictionary of token probabilities or frequency statistics from a large corpus. The surprisal of a sentence can be calculated as:

$$\text{Surprisal}(s) = - \sum_{w \in s} \log p(w)$$

where $p(w)$ is the probability of token w under the reference distribution. For our distribution, we utilize the word frequencies library in python, which obtains the frequencies of the top 100 million most popular words from multiple data sources, with a floor value given to missing words.

6.2.6 SurprisalLFU. Upon eviction, the item with the highest surprisal value out of all items with the minimal LFU counter is removed from the cache. This approach prioritizes retaining semantically common vectors that are more likely to contribute to future hits, while discarding outliers that are unlikely to be requested again. This is particularly useful, as we found that caches often accumulate a long tail of items with nearly identical counters.

6.2.7 SphereLFU. SphereLFU is an online policy designed to approximate the behavior of our best offline policy, FGRVB. Recall that FGRVB maximizes the total “volume” of future queries covered by the cache. Since SphereLFU lacks future knowledge, it approximates the total volume by the integral of the probability density function (PDF) of the request stream over the covered regions. Thus, SphereLFU reframes cache management as an online Kernel Density Estimation (KDE) task: by maintaining vectors that inhabit the highest-density regions of the embedding space, we maximize the expected number of future hits.

Rational. Standard frequency-based policies (like LFU) operate on a “winner-takes-all” basis: only the specific vector returned as a hit receives a frequency update. In a continuous semantic space, this is suboptimal. A query falling between two cached vectors v_1 and v_2 provides evidence that *both* are valuable, yet standard policies discard this information for the non-selected vector.

SphereLFU applies *fuzzy frequency updates*. We view each query not as a discrete point, but as a unit of probability mass diffusing into the local neighborhood. When a query arrives, we distribute this mass among all nearby cached vectors proportional to their proximity. This allows the cache to build a smooth density map of the query distribution, retaining groups of vectors that collectively cover high-traffic semantic regions, even if no single vector is an exact match for every query.

Probabilistic Model. We assume each request embedding q is generated by a latent semantic source Z , corresponding to one of the cached embeddings. Let $\{e_i\}$ denote the cached embeddings, all ℓ_2 -normalized. We model the conditional likelihood of observing q given source i using a spherical similarity kernel equivalent to the von Mises-Fisher distribution on the unit sphere:

$$p(q | Z = i) \propto \exp\left(-\frac{\kappa}{2} d(q, e_i)^2\right),$$

where $\kappa > 0$ is a concentration parameter determining the sharpness of the kernel.

Fuzzy Frequency Updates. Upon receiving a query q , we identify the set $\mathcal{M}(q)$ of cached embeddings within the threshold distance D_{thresh} . We calculate a *responsibility* score $r_i(q)$ for each neighbor $i \in \mathcal{M}(q)$, representing the probability that vector i is related to query q :

$$r_i(q) = \frac{(c_i + \alpha) \exp\left(-\frac{\kappa}{2} d_i^2\right)}{\sum_{j \in \mathcal{M}(q)} (c_j + \alpha) \exp\left(-\frac{\kappa}{2} d_j^2\right)}$$

where $d_i = d(q, e_i)$, c_i is the current accumulated frequency of embedding i , and $\alpha > 0$ is a smoothing constant. The counters are then updated by distributing exactly one unit of mass:

$$c_i \leftarrow c_i + r_i(q).$$

Unlike heuristic LFU counters, c_i here represents a sufficient statistic for the mixture weight $\pi_i = \Pr[Z = i]$, converging to the true usage density under stationary conditions.

Distribution Shift and Decay. To handle non-stationary workloads where topic popularity shifts over time, we apply a multiplicative decay factor $\gamma \in (0, 1)$ during updates: $c_i \leftarrow \gamma c_i + r_i(q)$.

This effectively creates an exponentially weighted moving average of the density. For computational efficiency, rather than decaying every counter on every step, we implement this lazily or by periodically rescaling all global counters by a fixed factor (e.g., halving all counters every N requests).

Efficiency. Unlike our other LFU variants that perform a k -NN search to update a single metadata entry, SphereLFU employs a range query to identify all vectors within the semantic threshold. While this theoretically allows for an update of the entire cache, the high-dimensional embedding space is characterized by extreme sparsity. In practice, the number of neighbors within the D_{thresh} values we experimented with is consistently low, fewer than 10. If the neighbor count were to increase significantly, it would indicate that D_{thresh} is likely set too high. In scenarios where strict latency bounds are required, computational complexity can be capped by restricting fuzzy frequency updates to the top k nearest neighbors. This approximation preserves the most significant portion of the probability mass while maintaining a guaranteed $O(k)$ update time.

Hyperparameter Configuration. The behavior of SphereLFU is governed by three primary parameters: the smoothing constant α , the kernel concentration κ , and the temporal decay factor γ . Based on empirical grid search tuning across the evaluation datasets, we set $\alpha = 10^{-3}$ and $\kappa = 2/D_{\text{thresh}}^2$. We set the decay parameter $\gamma = 1$, as the evaluated request traces did not exhibit significant distribution shifts or ordering bias. While this work utilizes static configurations for consistency across benchmarks, dynamic parameter tuning (for example, using stochastic hill climbing or ML methods) is a promising direction for adapting to changing workloads.

7 Experiments

In this section, we evaluate the performance of our proposed semantic caching policies against standard baselines as well as the offline oracles. We utilize the nine datasets described in Section 4, processing the first 100K entries of each (91K for NaturalQuestions). Based on the properties observed in our vector analysis and on Section 3.3, we set the semantic threshold distance $D_{\text{thresh}} = 0.9$ for all reported experiments. We selected the threshold such that the hit rate is within a useful range for all datasets, at least 10%, although the trends are similar for other thresholds. In Appendix B, we present results for the smaller normalized L2 distance thresholds of 0.7 and 0.5. In addition, the complete unprocessed data is available online [1].

7.1 Implementation and Hardware

We have implemented our code in Python, utilizing well known, open source modules such as NumPy, SciPy, etc for efficiency. For this paper, we utilized a Faiss flat index for our vector store, although our code also supports other vector stores such as Milvus and HNSWlib. Our code is available in open-source [1].

7.2 Hit Rate Results

The hit rate results are shown in Figure ?? . As can be seen, most datasets exhibit a strong frequency bias, which can be inferred from the 3 bands formed: (i) for offline policies, (ii) frequency based policies and (iii) the other temporal based policies. As presented,

LFU-based policies generally outperform recency-based methods such as FIFO and LRU. Among online policies, SphereLFU consistently achieves high hit rates, rivaling other frequency-based baselines. On “hard” datasets with long-tail distributions, such as StackOverflow and HotPotQA, SphereLFU remains the top online contender, though a noticeable gap remains between it and the theoretical upper bound obtained by the offline heuristics of VOPT.

This gap confirms that in low-signal regimes, clairvoyant knowledge provides a decisive advantage over online estimation. However, in non-stationary workloads like WildChat, high temporal locality allows LRU or time-decayed density (SphereLFU) to compete with the static volume maximization of FGRVB. The optimal online variant is highly workload-dependent: WildChat benefits from LFUDA’s aging, ELI5 from RAP’s probabilistic admission, and MMLU from LRU-K’s filtering of “one-hit wonders.” For the Quora dataset, which exhibits an order of magnitude lower hit rate, SurprisalLFU’s use of static linguistic features to identify cache-worthy prompts more effectively than frequency or recency alone.

FGRVB and RGRVB represent distinct optimization philosophies: maximizing total future coverage versus maximizing temporal precision. FGRVB treats caching as a Maximum Coverage Problem, effectively retaining “heavy hitter” vectors that cover high future volumes regardless of temporal distance. Conversely, RGRVB optimizes for the immediate next hit; while this excels in bursty workloads and avoids cache pollution, its “myopic” view causes it to plateau below FGRVB on static distributions.

Finally, CRVB attempts to reduce semantic caching to exact matching via pre-clustering. While theoretically sound, it struggles with the “overlapping cluster” phenomenon inherent to high-dimensional spaces. As D_{thresh} increases, queries often fall within the radii of multiple potential clusters, breaking the equivalence class assumption. Flexible greedy approaches (FGRVB and RGRVB) thus outperform CRVB in high-overlap regimes. In summary, FGRVB serves as the strongest approximation for the hit rate upper bound, while SphereLFU offers a balance between a high hit rate and good hit accuracy (presented in Section 7.3 below). At lower thresholds, SurprisalLFU benefits from breaking frequency ties using surprisal.

7.3 Semantic Accuracy

In semantic caching, a hit is not a binary event: the proximity of the retrieved vector to the query also indicates the quality of the hit[4]. We quantify this quality by using mean hit distance (MHD), the average distance between queries and their retrieved cached vectors. While measuring embedding similarity via cosine or L_2 distance is standard practice, we treat these metrics as a proxy for semantic fidelity. Consequently, a lower MHD indicates that the cache provides responses that are more useful for the user.

As shown in Figure 2, SphereLFU demonstrates a dominant advantage, achieving the lowest MHD across seven of the nine datasets. The exceptions, MMLU and WildChat, are characterized by a higher proportion of “one-hit wonders.” WildChat is heavily recency-biased due to iterative prompt reformulations, while MMLU’s extreme topical diversity creates a sparse environment where density estimation is less effective.

SphereLFU’s superior performance stems from its fuzzy frequency updates. By treating the cache as an online kernel density

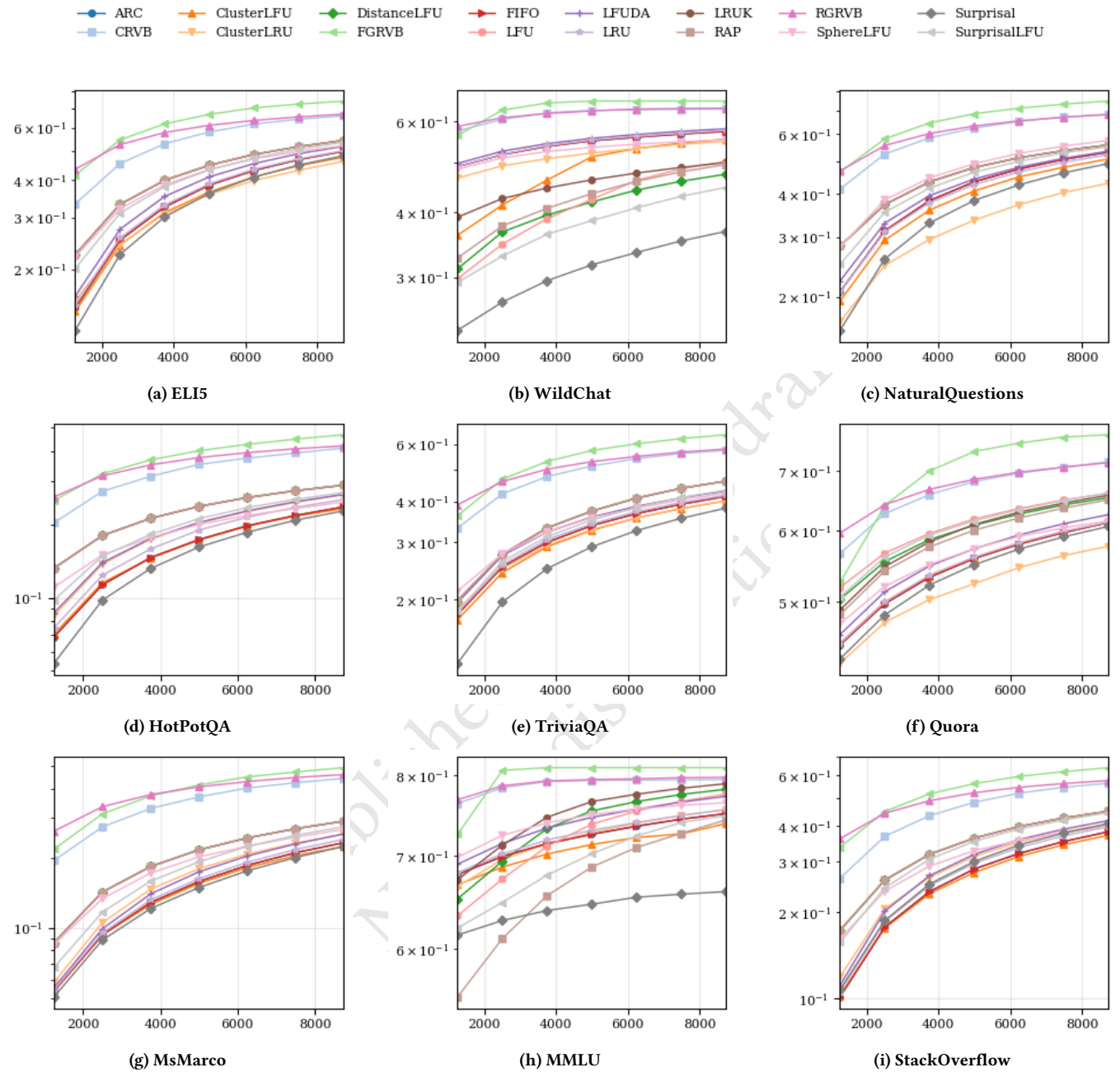


Figure 1: Hit rate as a function of cache size - note log scale on the Y-axis

estimator, it identifies and retains "prototypes" that serve as the medoids of high-density semantic regions. Notably, SphereLFU outperforms the three VOPT heuristics (CRVB, FGRVB, and RGRVB) in semantic accuracy. This results from their differing objective functions: while VOPT variants are strictly formulated to maximize the binary hit count, often by placing vectors at the fringes of clusters to maximize covered volume, SphereLFU naturally gravitates toward the center of request clusters.

These findings suggest that SphereLFU retains both high hit rates and good context integrity. This is critical for LLM-based systems and RAG pipelines, where the quality of the retrieved prompt or context directly impacts model performance. As the distance threshold increases, the advantage of SphereLFU becomes even more pronounced due to the higher density of hits per request.

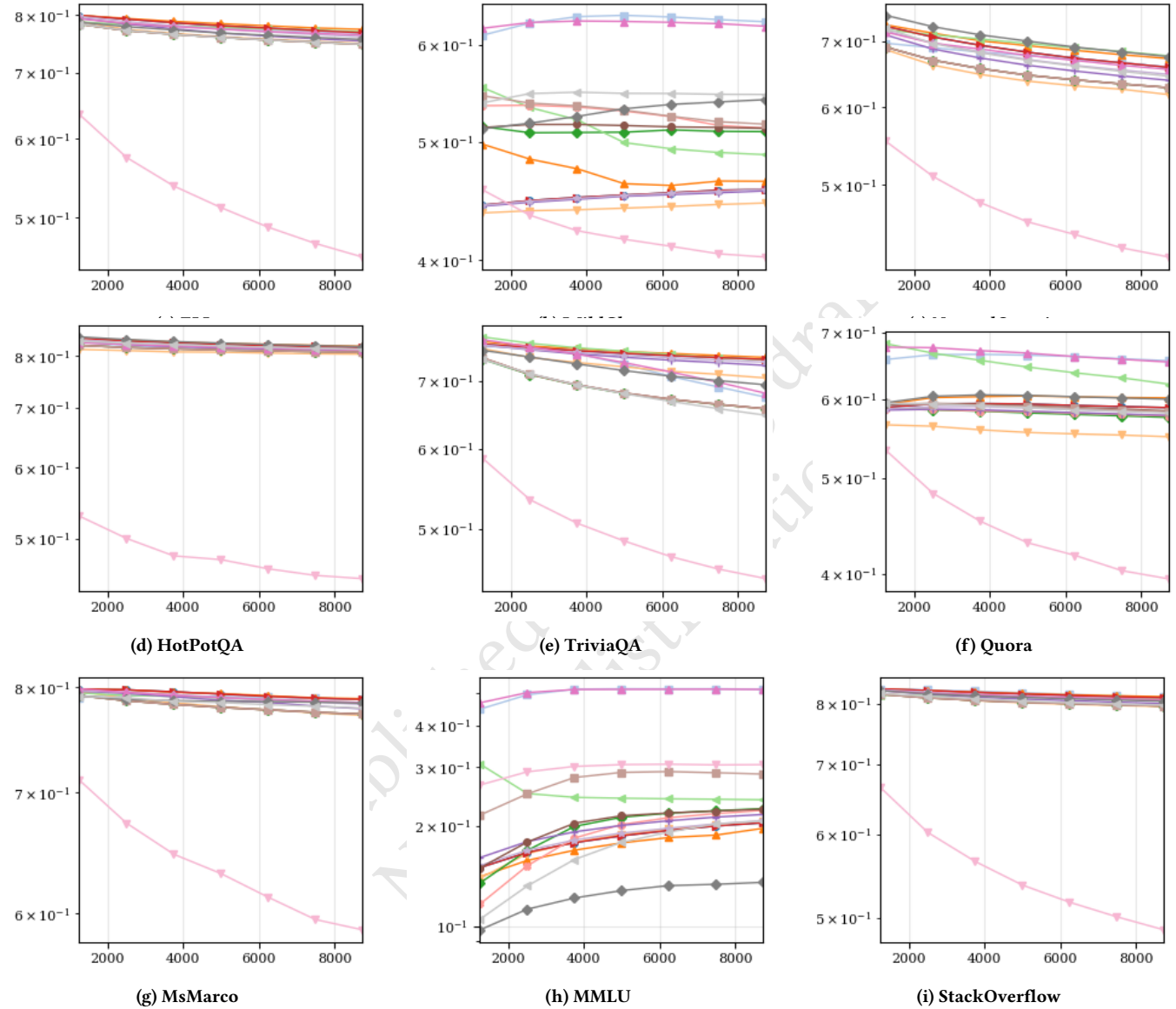
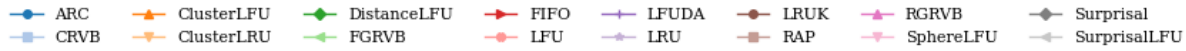


Figure 2: Mean hit distance as a function of cache size - note log scale on the Y-axis

8 Discussion

In this work, we have systematically studied replacement policies for semantic caching of LLMs and natural language based query-answering systems. Our study has revealed several gaps in transferring knowledge about caching from exact match caching to semantic caching. In particular, we identified that most of the tested workloads were frequency biased, so LFU variants performed best. We have also addressed the question of how to handle the book-keeping of cache management policies when there is more than

one possible hit in the semantic cache. We implemented several novel policies to evaluate different approaches.

We have also shown that computing VOPT, the adaptation of OPT for semantic caching, is NP-hard. Consequently, we introduced three heuristic alternatives to VOPT, named CRVB, FGRVB and RGRVB, and studied their performance. Our study has found that for most workloads, the best clairvoyant algorithm significantly outperform the best online policies, meaning that there is still room for additional research.

References

- [1] . 2026. Paper Implementation and Additional Data. (January 2026). -.
- [2] Martin Arltit, Rich Friedrich, and Tai Jin. 2000. Performance evaluation of Web proxy cache replacement policies. *Perform. Eval.* 39, 1-4 (Feb. 2000), 149–164.
- [3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2018. MS MARCO: A Human Generated MACHine Reading COMprehension Dataset. *arXiv preprint arXiv:1611.09268* (2018).
- [4] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. 212–218.
- [5] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving Cache Hit Rate by Maximizing Hit Density. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 389–403.
- [6] L. A. Belady and F. P. Palermo. 1974. On-line Measurement of Paging Behavior by the Multivalued MIN Algorithm. *IBM Journal of Research and Development* 18, 1 (1974), 2–19.
- [7] Ran Ben Basat, Xiaoqi Chen, Gil Einziger, Roy Friedman, and Yaron Kassner. 2019. Randomized Admission Policy for Efficient Top-k, Frequency, and Volume Estimation. *IEEE/ACM Transactions on Networking* 27, 4 (2019), 1432–1445.
- [8] Shai Aviram Bergman, Zhang Ji, Anne-Marie Kermarrec, Diana Petrescu, Rafael Pires, Mathis Randl, and Martijn de Vos. 2025. Leveraging Approximate Caching for Faster Retrieval-Augmented Generation. In *Proc. of the 5th Workshop on Machine Learning and Systems (EuroMLSys)*. 66–73.
- [9] Daniel Cer, Mona Diab, Iñigo Lopez-Gazpio, and Lucia Specia. 2017. SemEval-2017 Task 1: Semantic Textual Similarity - Multilingual and Cross-lingual Focused Evaluation. doi:10.18653/v1/S17-2001
- [10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [11] Gil Einziger, Ohad Eytan, Roy Friedman, and Ben Manes. 2018. Adaptive Software Cache Management. In *Proc. of the 19th International Middleware Conference (Middleware)*. 94–106.
- [12] G. Einziger, R. Friedman, and B. Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage (ToS)* 13, 4 (2017).
- [13] Angela Fan, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli. 2019. ELI5: Long Form Question Answering. In *Proc. of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 3558–3567.
- [14] Fănică Gavril. 1972. Algorithms for Minimum Coloring, Maximum Clique, Minimum Covering by Cliques, and Maximum Independent Set of a Chordal Graph. *SIAM J. Comput.* 1, 2 (1972), 180–187.
- [15] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. 2025. MeanCache: User-Centric Semantic Caching for LLM Web Services. In *Proc. of the IEEE 39th International Parallel and Distributed Processing Symposium (IPDPS)*.
- [16] Google. 2023. Stack Overflow (BigQuery Public Dataset). <https://huggingface.co/datasets/pacovaldez/stackoverflow-questions>. Google Cloud Public Datasets.
- [17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [18] John L. Hennessy and David A. Patterson. 2012. *Computer Architecture - A Quantitative Approach* (5. ed.). Morgan Kaufmann.
- [19] Dorit S. Hochbaum. 1996. *Approximating covering and packing problems: set cover, vertex cover, independent set, and related problems*. PWS Publishing Co., USA, 94–143.
- [20] Arun Iyengar, Ashish Kundu, Ramana Kompella, and Sai Nandan Mamidi. 2025. A Generative Caching System for Large Language Models. *arXiv #2503.17603*, <https://arxiv.org/abs/2503.17603>.
- [21] Shankar Iyer, Nikhil Dandekar, and Kornél Csernai. 2017. First Quora Dataset Release: Question Pairs. <https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>. Accessed 2025-09-10.
- [22] Song Jiang and Xiaodong Zhang. 2002. LIRS: An Efficient Low Inter-Reference Recency Set Replacement Policy to Improve Buffer Cache Performance. In *Proc. of the International Conference on Measurements and Modeling of Computer Systems SIGMETRICS*. 31–42.
- [23] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *arXiv #2404.12457*, <https://arxiv.org/abs/2404.12457>.
- [24] Theodore Johnson and Dennis Shasha. 1994. 2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm. In *Proc. of the 20th Int. Conf. on Very Large Data Bases (VLDB)*. 439–450.
- [25] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. *arXiv preprint arXiv:1705.03551* (2017).
- [26] G. Karakostas and D. N. Serpanos. 2002. Exploitation of Different Types of Locality for Web Caches. In *Proc. of the 7th Int. Symposium on Computers and Communications (ISCC)*.
- [27] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. https://direct.mit.edu/tacl/article/doi/10.1162/tacl_a_00276/43518.
- [28] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho, and Chong-Sang Kim. 2001. LRFU: A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies. *IEEE Trans. Computers* 50, 12 (2001), 1352–1361.
- [29] Cong Li. 2018. DLIRS: Improving Low Inter-Reference Recency Set Cache Replacement Policy with Dynamics. In *Proc. of the 11th ACM International Systems and Storage Conference (SYSTOR)*.
- [30] Nimrod Megiddo and Dharmendra S Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *2nd USENIX Conference on File and Storage Technologies (FAST)*. 115–130.
- [31] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical programming* 14, 1 (1978), 265–294.
- [32] Elizabeth J. O’Neil, Patrick E. O’Neil, and Gerhard Weikum. 1993. The LRU-K Page Replacement Algorithm for Database Disk Buffering. *ACM SIGMOD Rec.* 22, 2 (June 1993), 297–306.
- [33] Sejin Park and Chanik Park. 2017. FRD: A Filtering based Buffer Cache Algorithm that Considers both Frequency and Reuse Distance. In *Proc. of the 33rd IEEE International Conference on Massive Storage Systems and Technology (MSST)*.
- [34] Sajal Regmi and Chetan Phakami Pun. 2024. GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching. *arXiv #2411.05276*, <https://arxiv.org/abs/2411.05276>.
- [35] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proc. of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
- [36] Jon Saad-Falcon, Amanpreet Singh, Luca Soldaini, Mike D’Arcy, Arman Cohan, and Doug Downey. 2023. Embedding Recycling for Language Models. In *Findings of the Association for Computational Linguistics: EACL 2023*. 1936–1942.
- [37] Luis Gaspar Schroeder. 2024. VectorQ: Advanced Semantic Prompt Caching With Dynamic Thresholds and Performance-Based Clustering. *Technical University of Munich* (2024).
- [38] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2025. vCache: Verified Semantic Prompt Caching. *arXiv #2502.03771*, <https://arxiv.org/abs/2502.03771>.
- [39] Rana Shahout and Roy Friedman. 2024. Sketching the Path to Efficiency: Lightweight Learned Cache Replacement. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, Alysson Bessani, Xavier Défago, Junya Nakamura, Koichi Wada, and Yukiko Yamauchi (Eds.), Vol. 286.
- [40] Zhenyu Song, Daniel S Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 529–544.
- [41] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*.
- [42] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2369–2380.
- [43] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. 2024. WildChat: 1M ChatGPT Interaction Logs in the Wild. *arXiv preprint arXiv:2405.01470* (2024). <https://arxiv.org/abs/2405.01470> Dataset site: <https://wildchat.allen.ai>.
- [44] Banghua Zhu, Ying Sheng, Lianmin Zheng, Clark Barrett, Michael Jordan, and Jiantao Jiao. 2023. Towards Optimal Caching and Model Selection for Large Model Inference. *NIPS* 37 (2023).
- [45] Longwei Zou, Yan Liu, Jiamu Kang, Tingfeng Liu, Jiangang Kong, and Yangdong Deng. 2025. InstCache: A Predictive Cache for LLM Serving. *arXiv #2411.13820*, <https://arxiv.org/abs/2411.13820>.

	MsMarco	WildChat	ELI5	NQ	SO	Quora	MMLU	TriviaQA	HotPotQA
Cos Sim Avg	0.0182	0.0484	0.0523	0.0206	0.0170	0.0451	0.0562	0.0570	0.0848
Cos Sim Std	0.0757	0.0861	0.0845	0.0871	0.0893	0.0843	0.0995	0.0919	0.1011
L2 Mean	1.4019	1.3781	1.3754	1.3978	1.4019	1.3806	1.3718	1.3715	1.3508
L2 Std	0.0555	0.0655	0.0633	0.0644	0.0658	0.0635	0.0759	0.0687	0.0766
PCA Ent.	7.8781	7.8251	7.7055	7.6928	7.6043	7.7551	7.5655	7.5658	7.4615
Cluster AVG	1.0203	1.3606	1.0434	1.2347	1.0077	1.2958	1.9112	1.0716	1.0081
Cluster STD	0.1491	2.5079	0.2383	0.6622	0.0896	0.8880	1.3860	0.2975	0.0925
Hopkins	0.7401	0.8044	0.7558	0.7897	0.7424	0.8054	0.8839	0.7551	0.7252

Table 2: Per dataset metrics for first 100K embeddings in each dataset.

A Throughput Analysis

In fig. 8 we evaluate the system throughput by measuring the average operations per second (OPS) for the combined query and update cycle across the entire request stream. Since semantic caching inherently requires identifying similar vectors to determine hits, we treat the nearest neighbor (NN) search and the subsequent cache policy logic as an atomic operation.

Our profiling, as reported in Figure 8, indicates that the runtime is primarily dominated by the NN search latency, which is common to all semantic policies. Consequently, throughput remains consistent across different strategies, with all policies operating within the same order of magnitude. This suggests that the computational overhead introduced by the complex bookkeeping of policies like SphereLFU is negligible compared to the cost of the underlying vector retrieval.

To ensure scalability for larger cache sizes, we employed optimized implementations for all baselines. Notably, the offline clairvoyant policies (CRVB, FGRVB, RGRVB) achieve high throughput despite their algorithmic complexity. This is achieved by pre-computing a global pairwise distance matrix during initialization, effectively offloading the expensive distance calculations from the critical path of the evaluation.

FGRVB has the lowest throughput, which is caused by its implementation requiring a complex computation of the marginal gain for each vector per update.

B Additional Experiments

We provide supplemental evaluations for varying distance thresholds, specifically $D_{\text{thresh}} = 0.5$ and 0.7 (corresponding to cosine similarities of 0.88 and 0.76, respectively). These more conservative thresholds naturally result in fewer semantic hits per query. This shift diminishes the advantage of SphereLFU, which relies on a high density of neighboring vectors to effectively distribute soft-frequency updates. Conversely, SurprisalLFU exhibits relative robustness at lower thresholds. When most cached items share a baseline frequency of 1, its linguistic surprisal metric provides a more intelligent tie-breaking mechanism for eviction compared to random or recency-based selection.

C Datasets Analysis

C.1 Datasets Metrics

Table 2 presents key statistical properties of the embeddings (using the first 100K queries of each dataset) to characterize the geometric structure and clustering tendency of the semantic space. We consider the following metrics:

Cos Sim (Avg/Std): The average and standard deviation of cosine similarity between all pairs of embeddings. The low average values (near 0) indicate that the embedding space is sparsely populated, and random queries are typically orthogonal to one another.

L2 (Mean/Std): The average Euclidean distance between all embeddings pairs. Since our embeddings are normalized to the unit sphere, orthogonal vectors have a distance of $\sqrt{2} \approx 1.414$. The observed means ($\approx 1.37 - 1.40$) align with the cosine similarity results, confirming that unrelated queries are far apart in the vector space.

PCA Ent. (Entropy): A measure of the effective dimensionality of the data, calculated as the entropy of the explained variance ratio of the PCA components. A higher value indicates that information is distributed across many dimensions, whereas a lower value implies the data lies on a lower-dimensional manifold.

Cluster (AVG/STD): We grouped vectors into semantic clusters using the threshold $D_{\text{thresh}} = 1$. **Cluster AVG** denotes the average size of a cluster; values near 1.0 (e.g., StackOverflow) indicate high uniqueness, while higher values (e.g., MMLU) indicate high redundancy. **Cluster STD** measures the variance in cluster size; a high standard deviation (e.g., WildChat) indicates a long tail distribution where specific topics are extremely popular (highly repeated) while most others are unique.

Hopkins Statistic: A statistical test for spatial randomness. A value of 0.5 indicates a uniform (random) distribution, while values approaching 1.0 indicate a strong tendency to cluster. Our scores (ranging from 0.74 to 0.88) confirm that the datasets possess significant non-random semantic structure, justifying the use of semantic caching.

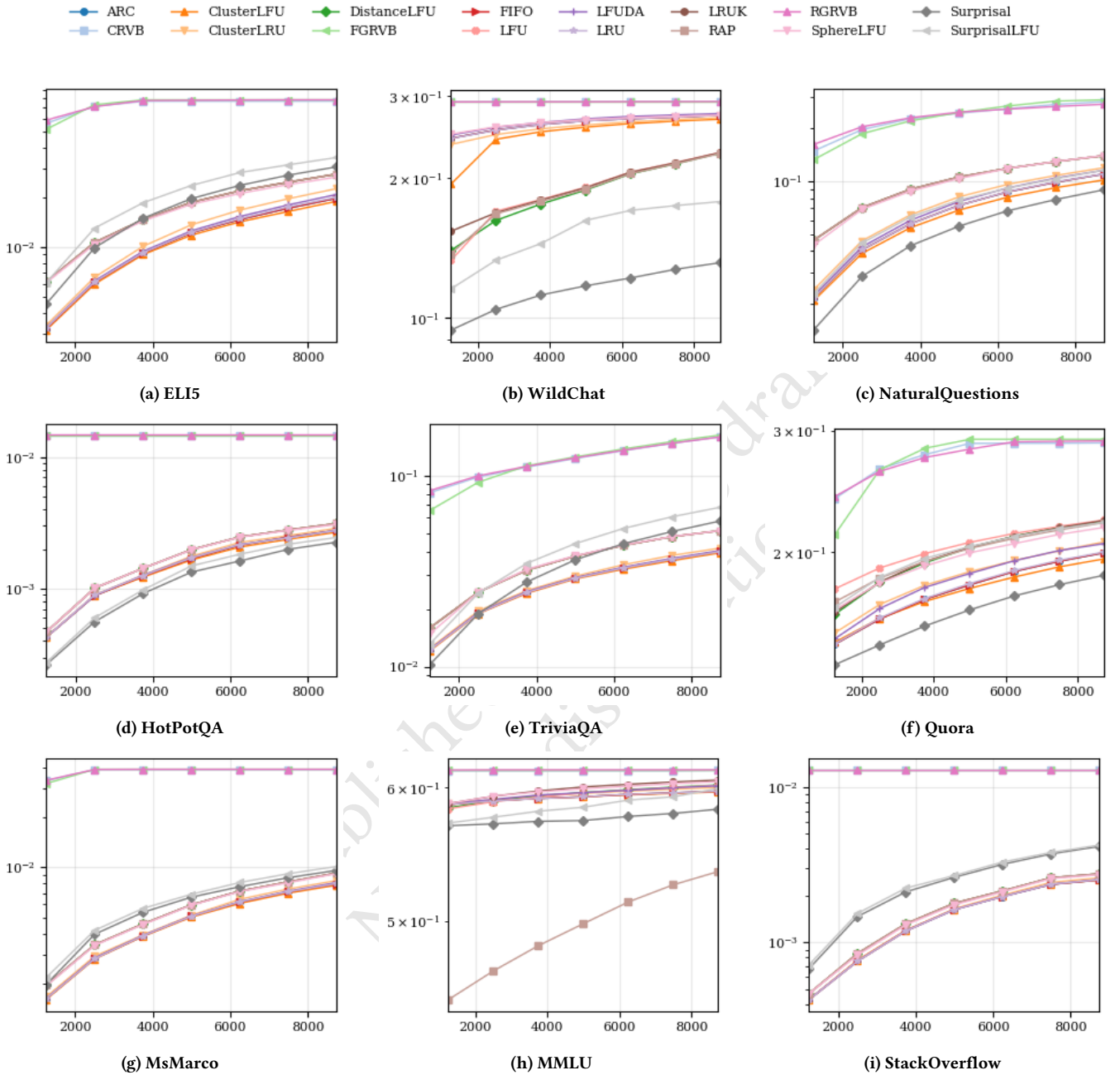


Figure 3: Hit rate at $D_{\text{thresh}} = 0.5$ as a function of cache size - note log scale on the Y-axis

C.2 Datasets Characteristic

The empirical distributions for MMLU and WildChat, as shown in Figure 7, diverge significantly from the other workloads, exhibiting a distinct "step" function across varying distance thresholds. Specifically, these datasets maintain a relatively flat density for small D_{thresh} values followed by a sharp slope. When cross-referenced with their exceptionally high cluster standard deviations, which are an order of magnitude larger than those of other datasets, this geometric profile suggests a high concentration of transient queries or "one-hit wonders."

While these workloads contain dense clusters of frequent queries, they are simultaneously characterized by a massive, sparse "long tail" of unique embeddings that do not overlap. For cache management, this indicates that while the "heavy hitters" are easily captured by

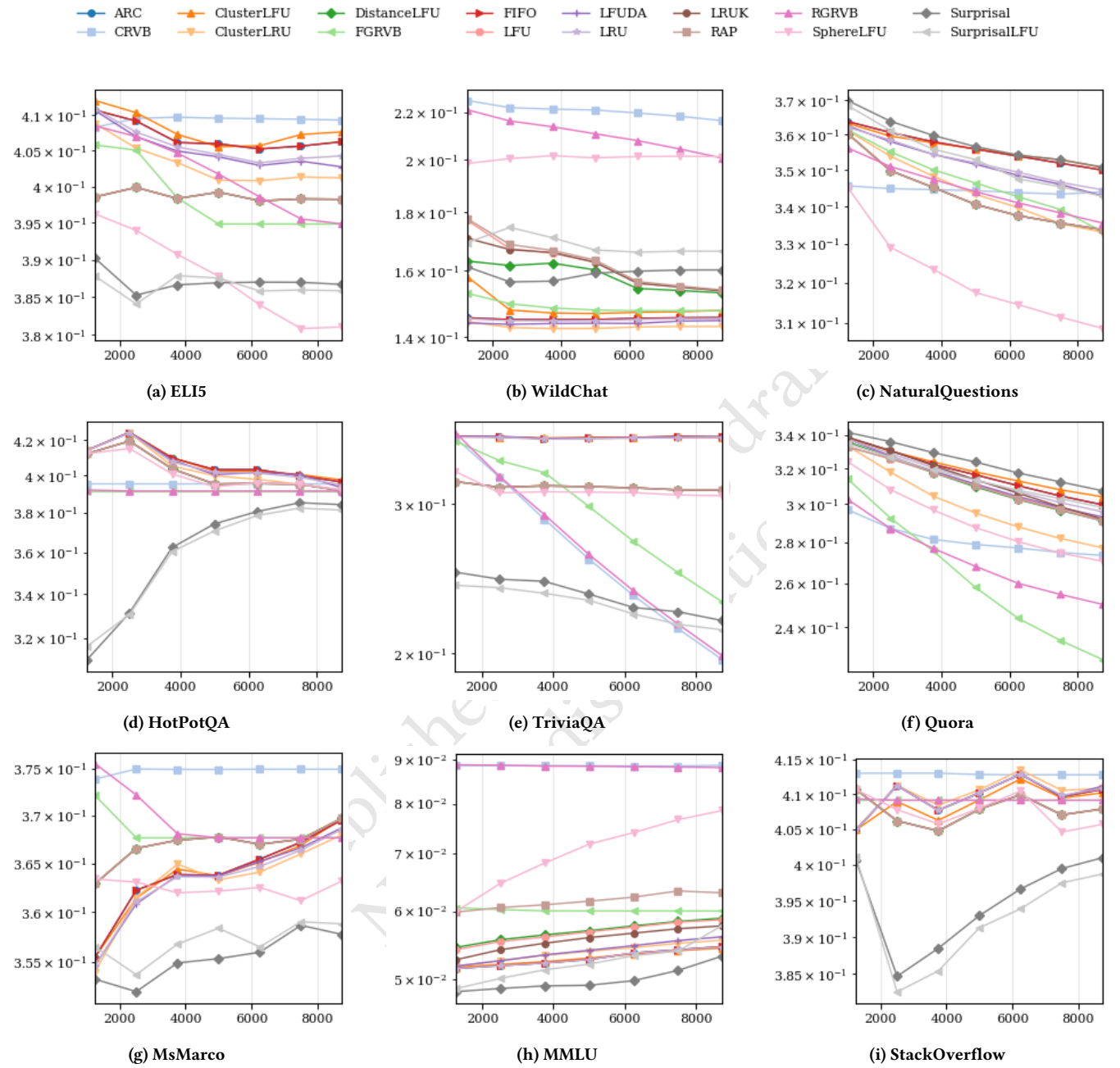


Figure 4: Mean hit distance at $D_{\text{thresh}} = 0.5$ Distance as a function of cache size - note log scale on the Y-axis

frequency-based policies, a substantial portion of the cache is at risk of pollution by these non-reusable entries unless an effective admission filter is employed.

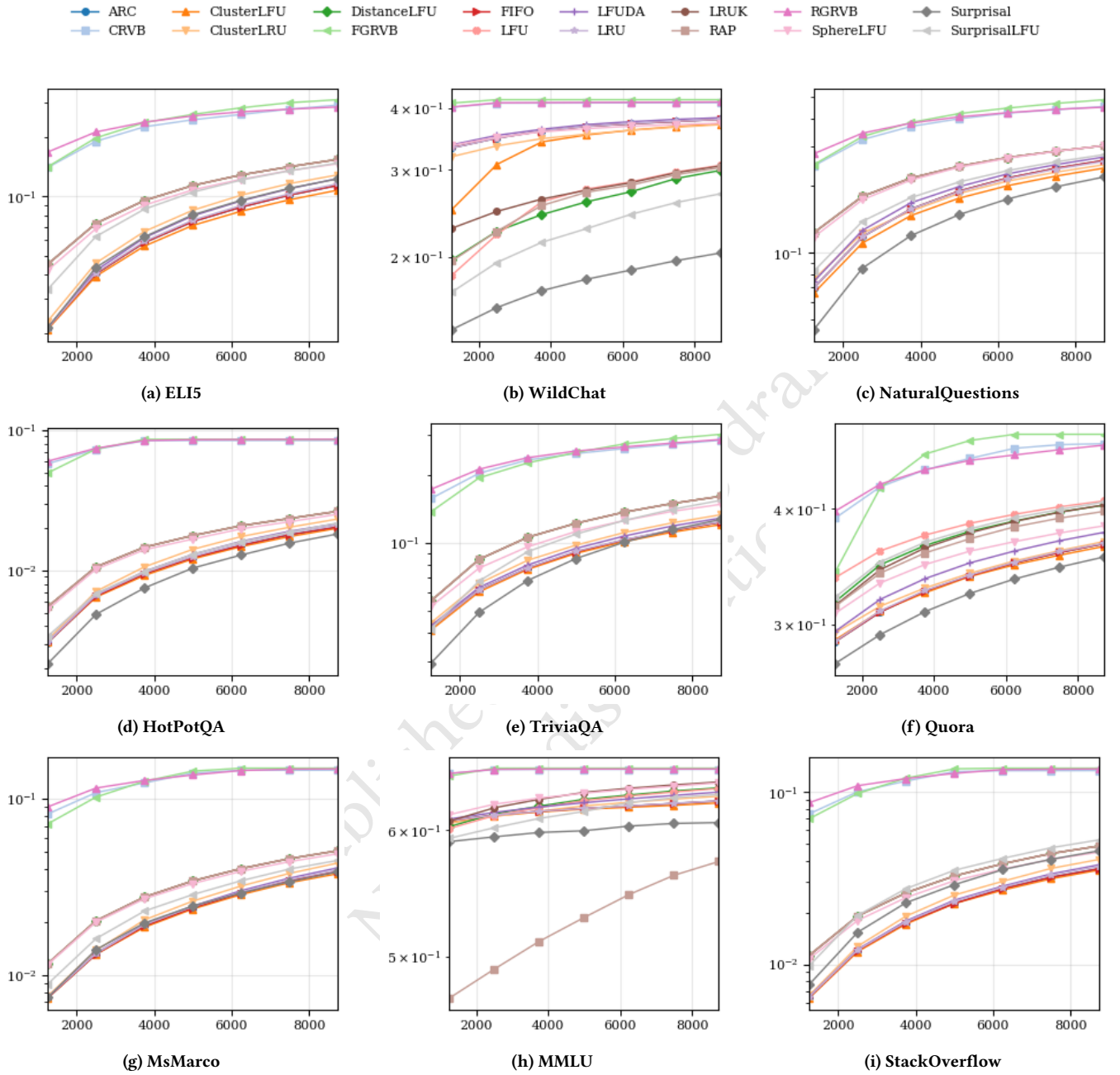
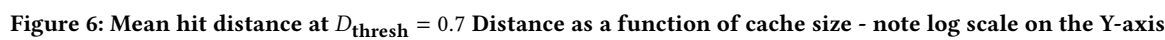


Figure 5: Hit rate at $D_{\text{thresh}} = 0.7$ as a function of cache size - note log scale on the Y-axis



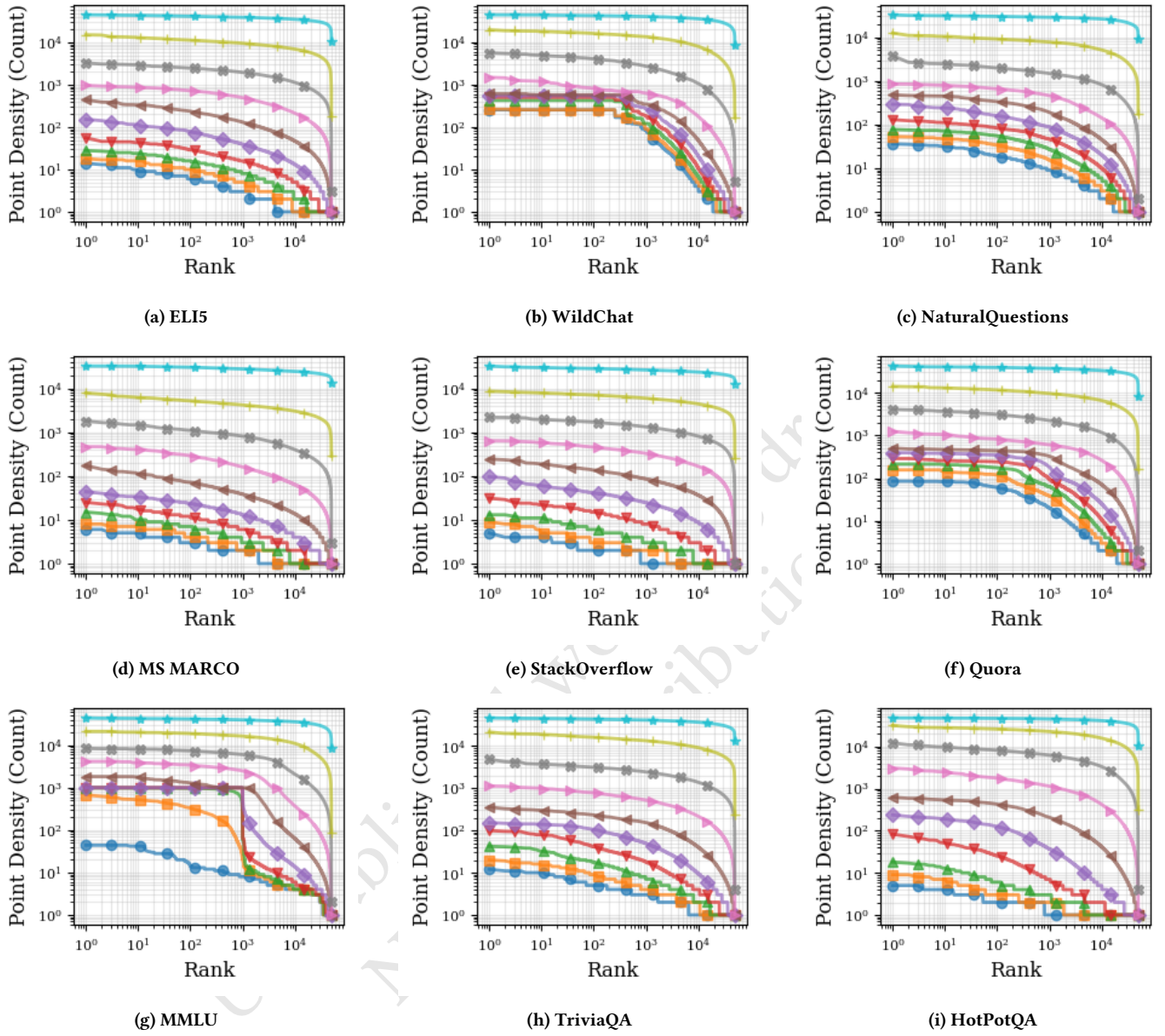


Figure 7: Point Density Distribution for Various Semantic Distance Threshold Values

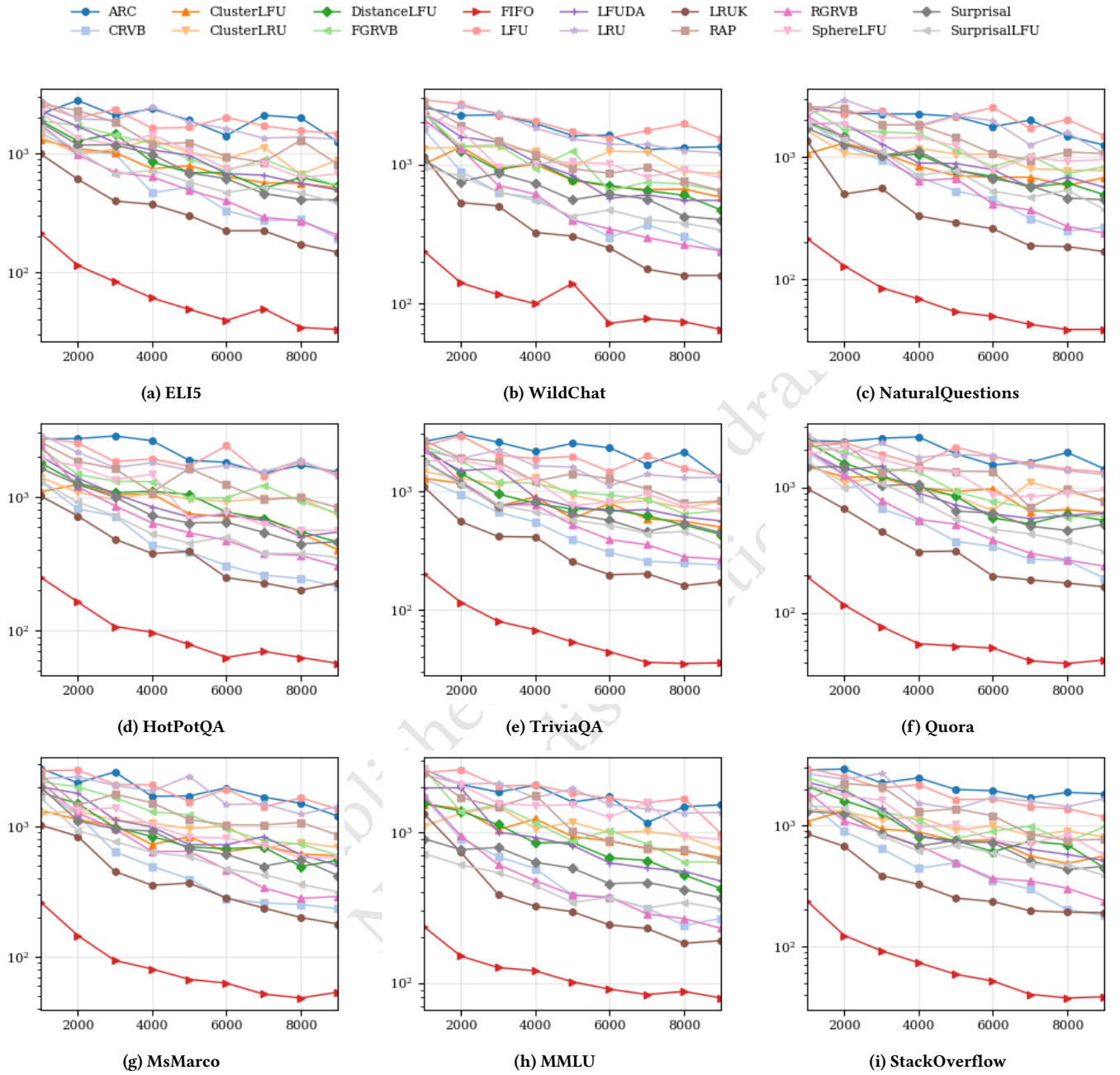


Figure 8: Throughput as a function of cache size - note log scale on the Y-axis